

JOCKY: An Evidence-Aware Investigation Compiler for Capability-Controlled, Provenance-Preserving Digital Forensics

Ayush Shukla

Thadomal Shahani Engineering College (TSEC)

Mumbai, India

hawwyush@gmail.com

Abstract—Digital forensic investigations today are assembled by hand from a patchwork of platform-specific scripts, endpoint query languages, collection tools and timeline analysers. The same question—“*did PowerShell drop an executable and then contact an external host?*”—must be rewritten for every operating system and evidence format, and the resulting evidence rarely carries a machine-checkable record of how it was obtained. We present JOCKY, an *investigation compiler*: the investigator writes *what* evidence and relationships are needed in a small declarative language, and the compiler decides *how, where and under which privileges* that evidence may be safely acquired. JOCKY introduces (i) an evidence-aware type system in which every value records its origin, collector and integrity; (ii) a two-level Forensic Intermediate Representation that separates investigation meaning from physical execution; (iii) static capability inference that computes the minimum endpoint privileges an investigation needs; (iv) a query-optimizer-style planner that pushes filters, joins and time windows down to the endpoint; and (v) a signed *Forensic Execution Contract* that makes each run auditable and replayable against frozen evidence. Rather than evading endpoint security, JOCKY is designed to *cooperate* with it: agents are signed, capabilities are declared up front and every action is logged. A prototype evaluation on synthetic telemetry from 10–100 hosts shows that the optimized plan transfers roughly 400× less data than naive full collection while producing a byte-identical set of findings, and that replaying the plan on preserved evidence reproduces the same findings digest.

Index Terms—digital forensics, incident response, domain-specific languages, compilers, intermediate representation, query optimization, capability-based security, provenance, WebAssembly, reproducibility

I. INTRODUCTION

When a security incident occurs, an investigator’s first job is to answer concrete questions about what happened on a set of computers: which program ran, what files it created, what network addresses it contacted, and in what order. This work is called *Digital Forensics and Incident Response* (DFIR). Although the questions are conceptually simple, answering them in practice is laborious. A Windows laptop records process activity through Event Tracing for Windows (ETW) and the Event Log [1]; a Linux server exposes it through `/proc`, the audit subsystem or eBPF [2]; a seized disk image or memory dump requires yet another toolchain such as Plaso [3] or Volatility [4]. Consequently an investigator often writes the *same* logical investigation several times—once in PowerShell, once in a shell script, once as a Velociraptor artifact [5], once

as an osquery SQL pack [6]—and then correlates the outputs by hand.

Two further problems compound this fragmentation. First, the collected evidence usually lacks a trustworthy *history*: which tool produced it, with what version and parameters, on which host, and whether it was transformed afterwards. Guidance such as NIST SP 800-86 [7] stresses exactly these chain-of-custody properties, yet most scripts do not record them. Second, investigators frequently collect far more than they need (“grab everything, sort it out later”), consuming endpoint CPU and network bandwidth and increasing the privacy footprint of the investigation.

This paper was motivated by Smart India Hackathon problem statement SIH26148 [8], which asks for a custom language and compiler for forensic programs that can execute despite endpoint security software. The problem statement lists techniques such as polymorphic code, custom encryption, in-memory execution and direct system calls. We deliberately take a different route. Those techniques optimise for *indistinguishability* from malware; a government or enterprise forensic platform needs the opposite—a *known* identity, *known* capabilities and a *known* provenance. Our guiding principle is therefore:

Move from security-control evasion to security-control cooperation: compile the *investigation*, not the executable.

Fig. 1 contrasts the two approaches. A traditional forensic script is compiled directly into an opaque binary. In JOCKY, investigator intent passes through semantic, capability, evidence and cost analyses before being lowered to a platform-specific plan, and the output is evidence bundled with its provenance.

Contributions. This paper makes the following contributions:

- 1) A reframing of forensic programming as *investigation compilation*, and the design of JOCKY, a small declarative language whose core operations (`observe`, `correlate`, `within`, `preserve`) carry forensic meaning rather than being syntactic sugar (§IV).
- 2) An *evidence-aware type system* (`Evidence<T>`, `Derived<T>`, `Finding`) that lets the compiler reject

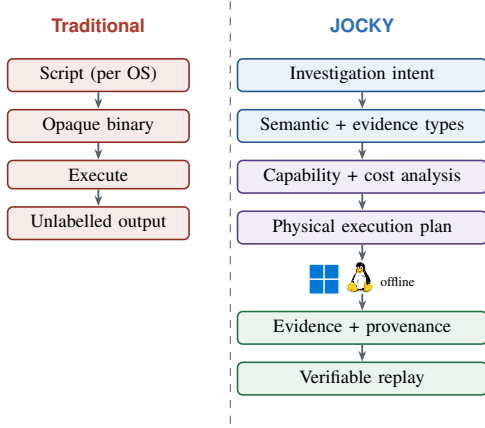


Fig. 1. Conventional forensic tooling compiles code; JOCKY compiles an investigation into an analysable, platform-specific plan whose output carries provenance.

investigations whose evidentiary guarantees cannot be met (§IV-A).

- 3) A two-level *Forensic IR* with static capability inference and a query-optimizer-style physical planner that targets Windows, Linux and offline evidence from one source (§V).
- 4) The *Forensic Execution Contract*, a signed, content-addressed bundle that makes execution auditable and replayable (§VI).
- 5) A prototype evaluation demonstrating a $\sim 400\times$ reduction in transferred data with identical findings and consistent replay (§VII).

II. BACKGROUND AND RELATED WORK

A. DFIR tooling

The DFIR ecosystem is mature, and JOCKY is designed to *complement* rather than replace it. Table I summarises the closest systems. Velociraptor’s VQL packages reusable “artifacts” with parameters and OS preconditions and deploys them as fleet-wide hunts [5]. osquery exposes the operating system as SQL tables with scheduled and distributed queries [6]. KAPE separates collection *targets* from processing *modules* for rapid triage [9]. Plaso builds “super-timelines” [3], [10] which Timesketch lets teams analyse collaboratively [11]. Volatility 3 provides plugin-based memory analysis [4], and Sigma offers a portable detection-rule format that now includes temporal and ordered correlations [12]. Each is excellent in its layer, but none takes a single declarative investigation and *compiles* it—with type, privilege and cost analysis—into plans for heterogeneous live and offline sources.

B. Forensic domain-specific languages

JOCKY is not the first forensic language, and we do not claim it is. Van den Bos and van der Storm introduced Derric, a DSL for describing file formats for data carving [15]. Most relevant is **Nugget** by Stelly and Roussev [13], [14], which proposed formal specification of forensic computations,

TABLE I
CLOSEST EXISTING SYSTEMS AND THE GAP JOCKY TARGETS

System	Already provides	Not its focus
Velociraptor (VQL)	Endpoint query language, reusable artifacts, fleet hunts	Static evidence types, capability inference, physical planning
osquery	SQL over OS state, distributed/scheduled packs	Cross-modality (disk, memory, PCAP) investigations
KAPE	Fast targeted collection + modules	Semantic model of the investigation
Plaso / Timesketch	Super-timelines, collaborative analysis	Controlling <i>how</i> evidence is acquired
Volatility 3	Memory forensics plugins	Live endpoint execution
Sigma	Portable detection rules, correlations	Collection, provenance, replay
Nugget [13]	Forensic DSL, auditable computation log	Live distributed endpoints, capability control

automatic optimised execution and a complete auditable log, identifying the lack of reproducibility and interoperability as the core problem of the field. JOCKY builds on this line of work and extends it into modern systems territory: live heterogeneous endpoints, least-privilege capability analysis, signed execution packages, sandboxed plugins, and provenance expressed in standard vocabularies.

C. Compiler, database and security foundations

Three well-established ideas underpin JOCKY. From **compilers**, we borrow multi-level IRs: MLIR supports domain-specific dialects that are progressively lowered [16], and LLVM provides mature native code generation [17]. From **databases**, we borrow cost-based query optimisation [18], [19]: distributed engines such as Trino split a query into stages executed near the data [20]. From **security**, we borrow capability-based access control [21], [22], as realised today by WebAssembly’s sandbox [23] and WASI’s “no ambient authority” design [24]; and supply-chain provenance, as in W3C PROV [25], in-toto [26] and Sigstore [27]. JOCKY’s contribution lies in combining these ideas specifically for forensic investigations.

III. DESIGN GOALS AND THREAT MODEL

We set six design goals, each tied to a measurable property:

- G1 Write once, run on many sources.** One investigation should run on Windows, Linux and offline evidence without source changes.
- G2 Least privilege.** The compiler must compute and declare the minimum capabilities required, and the runtime must enforce them.
- G3 Minimal collection.** Collect only the evidence the investigation logically needs (lower CPU, bandwidth and privacy cost).
- G4 Provenance by construction.** Every piece of evidence and every finding records how it was produced.
- G5 Replayability.** Given frozen evidence, re-running an investigation must reproduce identical findings.
- G6 Cooperation, not evasion.** JOCKY agents are signed, identifiable and allow-listable by endpoint detection and response (EDR) products.

Threat model. We assume a trusted control plane and investigator identity (e.g., via OIDC), an untrusted network, and potentially compromised endpoints. JOCKY protects the *integrity* of investigation packages and evidence (tampering is detected through signatures and hashes) and *limits the authority* of investigations and third-party plugins. It explicitly does *not* rely on process injection, API unhooking, vulnerable-driver loading, disabling EDR or covert command-and-control; such techniques are out of scope by design. A kernel-level adversary on the endpoint can still lie to any collector; JOCKY cannot prevent this but records which collector observed what, so that conflicting observations can be detected downstream.

IV. THE JOCKY LANGUAGE

JOCKY is intentionally small. It is not a general-purpose programming language; it describes investigations. Listing 1 shows our running example: find PowerShell processes that create a file and then open a network connection, all within 20 minutes.

```

1 investigation incident_42 {
2   scope hosts where environment == "lab";
3
4   let p = observe process
5     where image.name == "powershell.exe";
6   let f = observe file.create
7     where actor.pid == p.pid within 10m;
8   let n = observe network.connection
9     where actor.pid == p.pid within 20m;
10
11  correlate p -> f -> n within 20m as chain;
12
13  preserve chain with provenance integrity sha256;
14  emit chain;
15 }

```

Listing 1. A JOCKY investigation (running example).

Reading it line by line: `scope` selects which hosts or evidence sources participate; `observe` declares a *kind* of evidence—not the API that produces it; `where` filters it; `within` bounds time; `correlate a -> b` requires that events occur in order and share the stated keys; and `preserve` demands that the result be hashed and stored with its provenance. Notice what is *absent*: no Windows API names, no file paths to log files and no parsing code. The investigation says *what* is wanted and leaves *how* to the compiler, exactly as SQL leaves access paths to a database optimiser.

A. Evidence-aware types

JOCKY has ordinary types (String, Int, Timestamp, Duration, Hash, IP, Path), *semantic* types for forensic entities (Host, User, Process, File, NetworkConnection, RegistryKey), and—its key feature—*evidence* type constructors:

- $\text{Evidence}\langle T \rangle$: a raw observation of T , carrying *source*, *collector* (*name* + *version*), *host*, *event time*, *observation time* and a *content hash*.
- $\text{Derived}\langle T \rangle$: a value computed from evidence, carrying the identity of the transformation and the hashes of its inputs.

- $\text{Finding}\langle T \rangle$: a conclusion presented to a human, which must be traceable to evidence.

An $\text{Evidence}\langle \text{File} \rangle$ is therefore not merely a file: it is a file *plus a record of how we know about it*. Evidence values are immutable; a transformation never edits evidence but creates a new Derived value linked to its inputs. This enables compile-time checks that ordinary languages cannot express, for example: “*a Finding must depend only on Evidence that has an integrity hash*”, or “*this investigation correlates on event time, but the chosen offline source only provides observation time*”. Such programs are rejected, or flagged in the editor, before anything runs.

V. COMPILER ARCHITECTURE

Fig. 2 shows the complete system. We describe the compiler pipeline here and the runtime in §VI.

A. Front end

Source code is parsed with Tree-sitter [28], whose incremental concrete syntax trees serve both the compiler and the editor. The same front end powers a Language Server Protocol (LSP) implementation [29], which lets the IDE show *forensic* diagnostics rather than only syntax errors—for example: “*capability filesystem.content.read requested but not permitted by policy*”, “*finding depends on evidence without an integrity hash*”, or “*estimated upload exceeds 500 MB*”.

B. Two-level Forensic IR

The key architectural decision is to separate *what the investigation means* from *how it executes*. The **logical Forensic IR** is a small algebra of operators: Source, Observe, Filter, Project, Join, Window, Sequence, Correlate, Derive, Preserve, Attest and Emit. Listing 1 lowers to:

$$\text{Seq}_{\leq 20m}(\sigma_{img=ps}(\text{Obs}_{proc}) \bowtie_{pid} \text{Obs}_{file} \bowtie_{pid} \text{Obs}_{net})$$

The **physical Forensic IR** names concrete collectors, where each operator runs (endpoint or server), and how data is shipped. Fig. 3 illustrates how one logical plan yields three physical plans. For the minimum viable product the IR is serialised as JSON; its operator set maps naturally onto an MLIR dialect (`jocky.observe`, `jocky.join`, `jocky.window`, ...) [16] that can later be lowered through LLVM [17] to native code or to WebAssembly [23].

C. Static capability inference

Every sensitive operation implies a capability. Table II shows representative mappings. The compiler computes the required set for a program P as the union over its IR operators, $\text{Caps}(P) = \bigcup_{op \in \text{IR}(P)} \text{caps}(op)$, after optimisation, so that only operations that actually execute contribute. If $\text{Caps}(P) \not\subseteq \text{Policy}(\text{investigator}, \text{scope})$ the program is rejected before deployment. Because the set is computed statically and embedded in the signed contract, the endpoint runtime can enforce it and an EDR product can be told in advance exactly what the agent will and will not do—e.g., it may read process metadata and event logs, but it may not write memory, inject code or load drivers.

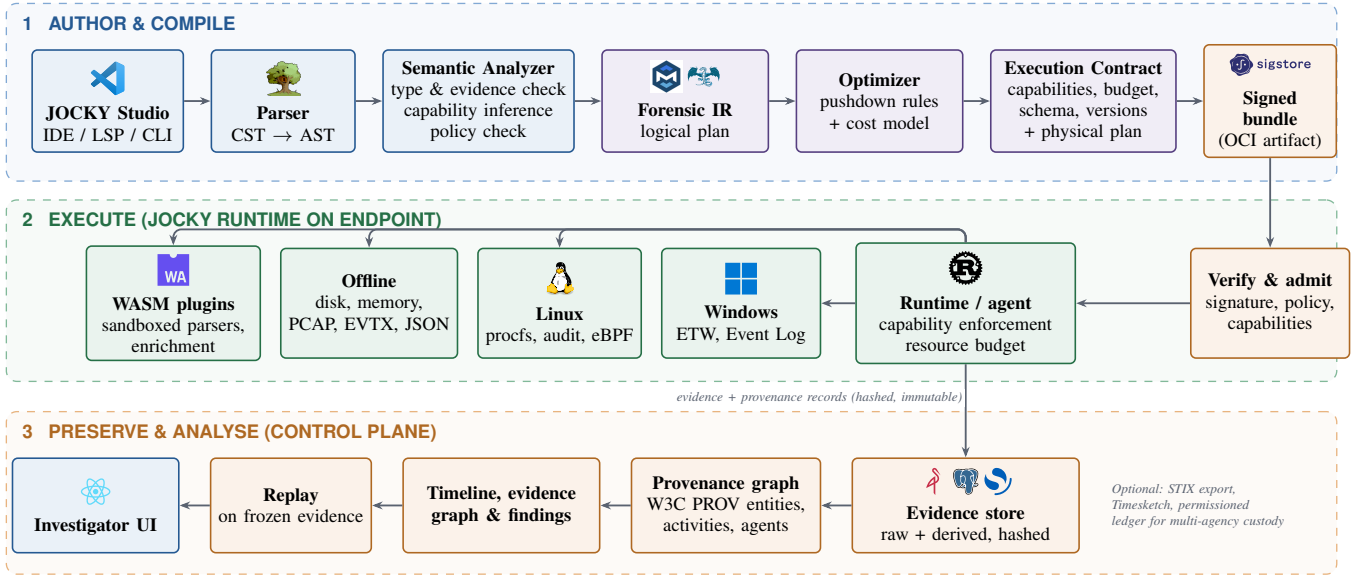


Fig. 2. JOCKY end-to-end architecture. Lane 1: the investigation is parsed, type- and capability-checked, lowered to a Forensic IR, optimised and sealed into a signed Execution Contract. Lane 2: the endpoint runtime verifies the bundle and dispatches only the declared, read-only collectors (native for OS telemetry, WebAssembly-sandboxed for third-party parsing). Lane 3: evidence is stored immutably with a provenance graph that feeds timelines, findings and replay. Logos are trademarks of their respective projects and denote the building blocks used.

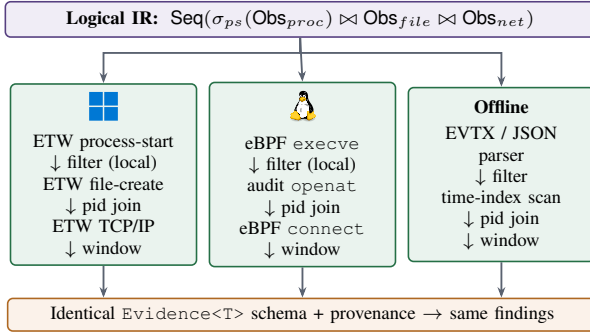


Fig. 3. Platform lowering: one logical plan, three physical plans. The investigation source is unchanged; only the collectors differ.

TABLE II
EXAMPLES OF JOCKY OPERATIONS AND INFERRED CAPABILITIES

Language construct	Inferred capability
observe process	process.observe
observe file.create	filesystem.metadata.read
reading file bytes / hashing content	filesystem.content.read
observe network.connection	network.observe
observe event_log	eventlog.read
observe memory.region	memory.read (high risk)

D. Forensic query optimisation

A naive way to run Listing 1 is to collect every process, file and network event from every host and correlate centrally. JOCKY instead applies rewrite rules familiar from relational optimisers [18], [19], adapted to evidence acquisition:

- **Predicate pushdown:** evaluate `image.name == "powershell.exe"` on the endpoint.
- **Semi-join pushdown:** keep the set of matching PIDs

locally and acquire only file and network events for those PIDs.

- **Temporal pushdown:** request only events inside the `within` windows relative to each process start.
- **Projection pushdown:** ship only fields referenced by the plan, plus the provenance fields required by `preserve`.
- **Acquisition sharing and collector selection:** serve several clauses from one event stream, and choose the cheapest collector that satisfies the required semantics.

Crucially, every rewrite must preserve *evidentiary* semantics, not just query results: a projection may never drop a field that a `preserve` or `Finding` depends on. This is where forensic optimisation differs from ordinary query optimisation—collection itself has a cost and perturbs the system being observed, so less collection is also better forensics.

VI. RUNTIME, PROVENANCE AND REPLAY

A. The Forensic Execution Contract

Compilation produces not just executable instructions but an explicit contract: the investigation intent and source hash; required evidence and capabilities; permitted collectors and platform constraints; a resource budget (CPU, memory, upload bytes); data-minimisation rules; integrity and provenance requirements; the output schema; and the versions of the compiler, IR, collectors and policy. The contract and physical plan are packaged as an OCI artifact [30], so existing container registries can distribute it, and signed with Sigstore/Cosign with an in-toto attestation [26], [27]. The resulting trust chain is: *investigator* → *source hash* → *compiler version* → *IR hash* → *plan hash* → *collector hashes* → *package signature* → *evidence hashes*. Altering any link invalidates the signature.

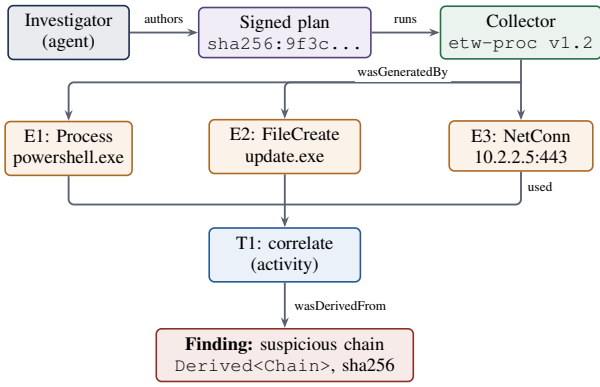


Fig. 4. Provenance graph for the running example (W3C PROV style). Every finding is traceable to raw evidence, the collector version and the signed plan that authorised its collection.

B. Endpoint runtime

The agent is written in Rust, whose ownership discipline eliminates common memory-safety bugs in a privileged, security-sensitive process [31]. On receipt of a bundle it verifies the signature, checks the declared capabilities against local policy, and dispatches read-only native collectors (ETW and Event Log on Windows [1]; procfs, audit and eBPF on Linux [2]; or file parsers for offline evidence). Third-party parsers and enrichment functions run as WebAssembly components in Wasmtime [32]. Following WASI’s capability model [24], a plugin receives only the handles it is explicitly granted—for example, a stream of already-collected records—and cannot open files or sockets on its own. We deliberately do *not* treat WebAssembly as the privileged collection boundary: sandboxing restricts code, it does not grant it access to the registry or the kernel. The division is therefore *privileged native adapter* → *controlled interface* → *sandboxed plugin*.

C. Provenance graph

Each collected record becomes an immutable, content-addressed `Evidence` entity; each transformation is an activity; each collector, plugin and investigator is an agent, following the W3C PROV data model [25]. Fig. 4 shows the graph for our running example. Any finding can be traced back, edge by edge, to the raw observations and the exact software that produced them. Normalised events borrow OpenTelemetry’s distinction between *event time* and *observed time* [33], but the original raw record is always retained by reference so that no forensic detail is lost in normalisation. Findings can be exported to STIX 2.1 [34] for threat-intelligence sharing.

D. Replay and forensic equivalence

Live systems change continuously, so perfectly re-running a live investigation is impossible. JOCKY therefore distinguishes *live execution* from *replayable execution*: replay is defined only against a *frozen* evidence source (preserved records, disk or memory image). We call two executions *forensically equivalent* if they have (a) the same input evidence hashes, (b) the same evidence lineage, and (c) the same canonical set

of findings—even if physical operator order, collector implementation or serialisation differ. A replay report checks each property (input hashes, collector and plan versions, lineage, findings) and marks it PASS or FAIL. This definition is what allows the optimiser to change *how* an investigation runs while guaranteeing that *what* it concludes is unchanged.

E. Why not a blockchain?

Because SIH26148 sits under a blockchain and cybersecurity theme, it is tempting to record evidence on a blockchain. The actual requirement, however, is *tamper-evident, verifiable provenance*, which SHA-256 content addressing, Merkle trees, signed attestations and a transparency log (Sigstore’s Rekord [27]) already provide without per-event consensus cost. A permissioned ledger remains a reasonable *optional* back end for multi-agency chain of custody, but it is not the core storage mechanism.

VII. PROTOTYPE AND PRELIMINARY EVALUATION

A. Setup

To test the central hypothesis—*collection can be optimised like a database query while preserving evidentiary results*—we built a Python prototype of the planner and ran Listing 1 under two physical plans. We generated synthetic endpoint telemetry for fleets of 10, 50 and 100 Windows-style hosts. Each host has 800 processes drawn from a realistic mix of 13 images (`svchost`, `chrome`, ..., with `powershell.exe` at about 2%), each spawning 0–12 file-creation and 0–6 network events with full field sets (command line, user, hashes, sizes, addresses). One in five hosts additionally receives an injected malicious chain (PowerShell → `update.exe` → connection to 10.2.2.5 within 20 min). The **naive** plan ships all events with all fields, serialised as JSON, and correlates centrally. The **optimised** plan applies predicate, semi-join, temporal and projection pushdown on each endpoint. For both plans we measure bytes and records transferred, central correlation time and a SHA-256 digest of the canonical findings list. Random seeds are fixed so the experiment is reproducible.

B. Results

Table III summarises the results. Across all fleet sizes the optimised plan transfers roughly **400× fewer bytes** and about **300× fewer records**, while the findings digests of the naive and optimised plans are *identical*. The number of findings exceeds the number of injected chains because benign PowerShell activity occasionally matches the same pattern—an expected property of a behavioural query, which both plans report identically. Re-running the optimised plan on the frozen dataset produced the same digest in every case, satisfying the replay criterion. Central correlation time also fell by one to two orders of magnitude, since the server receives only candidate records.

Ablation. Fig. 5 applies the rewrite rules cumulatively on the 100-host fleet. Predicate pushdown alone helps little (1.1×), because process records are a small share of the volume. The decisive step is the *PID semi-join* (50×): once the

TABLE III
NAIVE FULL COLLECTION VS. JOCKY-OPTIMISED PLAN (SYNTHETIC
FLEET, 800 PROCESSES/HOST)

Hosts	Naive		Optimised		Red.	Same findings	Replay digest
	MB	records	KB	records			
10	16.15	79,776	39.4	270	410×	✓ (12)	✓
50	80.85	399,496	197.4	1,332	410×	✓ (81)	✓
100	162.07	800,539	402.4	2,707	403×	✓ (143)	✓

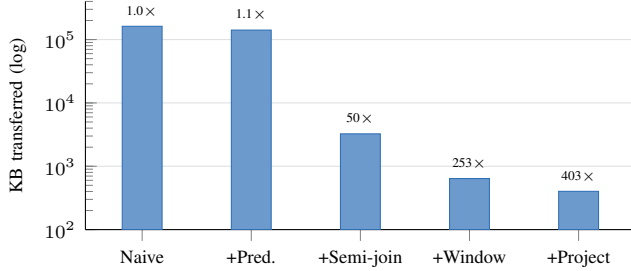


Fig. 5. Cumulative effect of each optimisation rule on data transferred (100 hosts). Labels show reduction relative to naive collection. Findings are identical for every bar.

endpoint knows which processes matter, it can ignore almost all file and network activity. Temporal pushdown adds a further 5×, and projection pushdown 1.6×. The findings digest was identical at every step, confirming that each rule preserved the investigation’s meaning.

C. Planned evaluation

The prototype isolates the planner; the full evaluation plan measures the whole system on real collectors: (1) *authoring*—time, lines of code and platform-specific code for five standard investigations written in JOCKY, VQL, osquery SQL and Python; (2) *optimisation*—endpoint CPU seconds, peak memory, bytes and latency on live Windows and Linux lab hosts; (3) *portability*—percentage of source unchanged across Windows, Linux and offline evidence; (4) *replay*—lineage and findings equivalence across repeated runs; and (5) *security*—rejection of altered packages, invalid signatures, undeclared capabilities, out-of-policy collectors and WASM plugins requesting forbidden capabilities.

VIII. DISCUSSION AND LIMITATIONS

Synthetic data. Our preliminary numbers come from synthetic telemetry with a stylised event distribution. Real reduction ratios depend on the selectivity of each investigation: a query that matches many processes will benefit less. The ablation, however, shows *where* the benefit comes from (joins and time windows), which we expect to generalise.

Endpoint-side computation. Pushdown moves work to endpoints. JOCKY bounds this through the contract’s resource budget, and the cost model can choose to correlate centrally when an endpoint is constrained.

Observer effect and trust. Collection perturbs live systems, and a compromised kernel can deceive any collector. JOCKY

does not solve these fundamental problems, but it makes them visible by recording exactly which collector observed which fact and when.

Adoption. JOCKY must be additive before it is transformational. We envisage a staged path: first, JOCKY compiles to existing Velociraptor artifacts and osquery packs; next, native collectors fill gaps; finally, organisations share signed, standardised investigations through a registry. AI assistance, if used, sits *after* deterministic evidence collection (e.g., clustering or anomaly scoring over the evidence graph), and its outputs are recorded as *Derived* values with model version, inputs and confidence, preserving auditability.

IX. FUTURE WORK AND CONCLUSION

Near-term work includes a minimum viable product with eight primitives (observe, filter, select, join, within, sequence, preserve, emit), a five-entity schema (Process, File, NetworkConnection, User, Event), Windows and Linux collectors and an offline JSON/CSV adapter. Longer-term directions include an MLIR dialect and LLVM/WebAssembly back ends; adapters for Plaso, Volatility and KAPE; Sigma import; Timesketch and STIX export; and a full cost-based distributed optimiser. The deeper research questions are whether a type-and-effect system can soundly track evidence origin and capability, and how to formally define and verify forensic equivalence between logical and physical plans.

In summary, JOCKY treats a forensic investigation as a program with meaning that a compiler can analyse. By separating *what* an investigator needs from *how* it is collected, JOCKY can compute least-privilege capabilities, optimise collection by orders of magnitude without changing conclusions, run the same investigation across operating systems and evidence formats, and attach verifiable provenance to every result. The one-line summary of the idea is: **JOCKY does not compile code; it compiles investigations.**

ACKNOWLEDGMENT

The author thanks the maintainers of the open-source projects on which this design builds. All logos shown in the figures are trademarks of their respective owners and are used only to identify the corresponding technologies.

REFERENCES

- [1] Microsoft, “About event tracing,” *Microsoft Learn*, Win32 documentation. [Online]. Available: <https://learn.microsoft.com/windows/win32/etw/about-event-tracing>
- [2] eBPF Foundation, “eBPF documentation.” [Online]. Available: <https://ebpf.io>
- [3] Plaso Project, “Plaso (log2timeline) documentation.” [Online]. Available: <https://plaso.readthedocs.io>
- [4] Volatility Foundation, “Volatility 3 documentation.” [Online]. Available: <https://volatility3.readthedocs.io>
- [5] Velocidex, “VQL—Velociraptor documentation.” [Online]. Available: <https://docs.velociraptor.app/docs/vql/>
- [6] osquery Project, “osquery configuration and deployment.” [Online]. Available: <https://osquery.readthedocs.io>
- [7] K. Kent, S. Chevalier, T. Grance, and H. Dang, “Guide to integrating forensic techniques into incident response,” NIST Special Publication 800-86, 2006.

- [8] Smart India Hackathon 2026, “Problem statement SIH26148,” Ministry of Education’s Innovation Cell, Government of India, 2026.
- [9] E. Zimmerman, “KAPE documentation.” [Online]. Available: <https://ericzimmerman.github.io/KapeDocs/>
- [10] K. Guðjónsson, “Mastering the super timeline with log2timeline,” SANS Institute InfoSec Reading Room, 2010.
- [11] Google, “Timesketch: Collaborative forensic timeline analysis.” [Online]. Available: <https://github.com/google/timesketch>
- [12] SigmaHQ, “Sigma rules and correlation specification.” [Online]. Available: <https://sigmahq.io/sigma-specification/>
- [13] C. Stelly and V. Roussev, “Nugget: A digital forensics language,” *Digital Investigation*, vol. 24, pp. S38–S47, 2018.
- [14] C. D. Stelly, “A domain specific language for digital forensics and incident response,” Ph.D. dissertation, Univ. of New Orleans, 2019.
- [15] J. van den Bos and T. van der Storm, “Bringing domain-specific languages to digital forensics,” in *Proc. 33rd Int. Conf. Software Engineering (ICSE)*, 2011, pp. 671–680.
- [16] C. Lattner *et al.*, “MLIR: Scaling compiler infrastructure for domain specific computation,” in *Proc. IEEE/ACM CGO*, 2021, pp. 2–14.
- [17] C. Lattner and V. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *Proc. IEEE/ACM CGO*, 2004, pp. 75–86.
- [18] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price, “Access path selection in a relational database management system,” in *Proc. ACM SIGMOD*, 1979, pp. 23–34.
- [19] G. Graefe, “The Cascades framework for query optimization,” *IEEE Data Eng. Bull.*, vol. 18, no. 3, pp. 19–29, 1995.
- [20] Trino Project, “Trino overview: Query execution model.” [Online]. Available: <https://trino.io/docs/current/overview.html>
- [21] J. B. Dennis and E. C. Van Horn, “Programming semantics for multi-programmed computations,” *Commun. ACM*, vol. 9, no. 3, pp. 143–155, 1966.
- [22] M. S. Miller, “Robust composition: Towards a unified approach to access control and concurrency control,” Ph.D. dissertation, Johns Hopkins Univ., 2006.
- [23] A. Haas *et al.*, “Bringing the web up to speed with WebAssembly,” in *Proc. ACM PLDI*, 2017, pp. 185–200.
- [24] WebAssembly Community Group, “WASI design principles.” [Online]. Available: <https://github.com/WebAssembly/WASI>
- [25] L. Moreau and P. Missier (Eds.), “PROV-DM: The PROV data model,” W3C Recommendation, 2013.
- [26] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappsos, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in *Proc. 28th USENIX Security Symp.*, 2019, pp. 1393–1410.
- [27] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software signing for everybody,” in *Proc. ACM CCS*, 2022, pp. 2353–2367.
- [28] M. Brunsfeld *et al.*, “Tree-sitter: An incremental parsing system.” [Online]. Available: <https://tree-sitter.github.io>
- [29] Microsoft, “Language Server Protocol specification 3.18.” [Online]. Available: <https://microsoft.github.io/language-server-protocol/>
- [30] Open Container Initiative, “OCI image manifest specification.” [Online]. Available: <https://specs.opencontainers.org/image-spec/manifest/>
- [31] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, “RustBelt: Securing the foundations of the Rust programming language,” *Proc. ACM Program. Lang.*, vol. 2, no. POPL, pp. 66:1–66:34, 2018.
- [32] Bytecode Alliance, “Wasmtime documentation.” [Online]. Available: <https://docs.wasmtime.dev>
- [33] OpenTelemetry, “Semantic conventions for events.” [Online]. Available: <https://opentelemetry.io/docs/specs/semconv/>
- [34] OASIS, “STIX Version 2.1,” OASIS Standard, 2021.